Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет информационных технологий и управления
Кафедра информационных технологий автоматизированных систем

ОТЧЕТ
по контрольной работе № 2

по дисциплине
«ИТиВП»

на тему «Интеграция и оптимизация»

Выполнил: студент группы 320601
А. С. Образцов

Проверила: Н. В. Хаджинова

Минск 2026

1 ЦЕЛЬ РАБОТЫ

Целью контрольной работы является интеграция клиентского приложения на *React* с серверной частью на *Node.js*/*Express*, оптимизация производительности клиента и подготовка развёртывания в облаке. Предметная область — сервис планирования задач по методу *GTD* (*Getting Things Done*) с контекстами и напоминаниями (вариант 27). Серверная часть (*REST API* с *JWT*, *PostgreSQL*, ролями *user*, *moderator*, *admin*) разработана в контрольной работе 1; в этой работе к ней добавлен клиент в папке *client/*, а основной сущностью интерфейса является задача (*Task*): список, страница задачи, форма создания и редактирования.

2 ССЫЛКА НА ПРИЛОЖЕНИЕ

Развёрнутое приложение: фронтенд — [будет заполнено после развёртывания], бэкенд — [будет заполнено после развёртывания]. Исходный код размещён в репозитории *https://github.com/Alexandr-Obraztsov/gtd-task-service*, ветка *kr22*. Конфигурации развёртывания (*render.yaml*, *client/vercel.json*) подготовлены и описаны в подразделе 3.8.

3 ХОД РАБОТЫ

3.1 Интеграция клиента и сервера

Клиент создан на *Vite* с библиотеками *React*, *React Router* и *axios*. Состояние хранится в хуках *useState*, *useEffect* и *useReducer*, готовых менеджеров состояния и кэша не используется. Запросы к *REST API* собраны в отдельный слой *src/api*: единый экземпляр *axios* (*http.js*), функции авторизации (*auth.js*) и по одной функции на операцию с задачами (*tasks.js*). Страницы не знают адресов и заголовков: они вызывают *listTasks*, *getTask*, *createTask*, *replaceTask* (метод *PUT*), *patchTask* и *deleteTask*. Базовый адрес берётся из переменной окружения *VITE_API_URL*, которую *Vite* подставляет при сборке.

Маршруты клиента: главная (*/*), вход и регистрация (*/login*, */register*), список задач (*/tasks*), форма создания (*/tasks/new*), страница задачи (*/tasks/:id*) и форма редактирования (*/tasks/:id/edit*). Маршруты задач обёрнуты в компонент *ProtectedRoute*, который отправляет неавторизованного пользователя на страницу входа.

3.2 Сервис *axios*, токены и обновление сессии

Access”=токен хранится только в памяти модуля, а не в *localStorage*, поэтому скрипт, прочитавший хранилище, токен не получит:

```
let accessToken = null;

export function getAccessToken() { return accessToken; }
export function setAccessToken(token) { accessToken = token; }
export function clearAccessToken() { accessToken = null; }
```

Refresh”=токен сервер кладёт в *httpOnly cookie*, клиент его не видит; чтобы браузер отправлял *cookie* на другой источник, экземпляр *axios* создаётся с *withCredentials*. Перехватчики добавляют заголовок *Authorization* и при ответе *401 TOKEN_EXPIRED* один раз обновляют сессию и повторяют запрос:

```
export const http = axios.create({
  baseURL: API_URL,
  withCredentials: true,
  timeout: 15000,
});

let refreshPromise = null;
```

```
export function refreshSession() {
  if (!refreshPromise) {
    refreshPromise = http
      .post('/auth/refresh', null, { skipAuthRefresh: true })
      .then(({ data }) => {
        setAccessToken(data.accessToken);
        return data;
      })
      .finally(() => { refreshPromise = null; });
  }
  return refreshPromise;
}

http.interceptors.request.use((config) => {
  const token = getAccessToken();
  if (token) config.headers.Authorization = 'Bearer ' + token;
  return config;
});

http.interceptors.response.use(
  (response) => response,
  async (error) => {
    const { config, response } = error;
    const expired = response?.status === 401
      && response.data?.error?.code === 'TOKEN_EXPIRED';
    if (!expired || !config || config.skipAuthRefresh
        || config.retried) {
      return Promise.reject(error);
    }
    try {
      await refreshSession();
    } catch (refreshError) {
      clearAccessToken();
      onSessionExpired();
      return Promise.reject(error);
    }
    return http({ ...config, retried: true });
  },
);
```

Запрос обновления общий для всех одновременных вызовов (*refreshPromise*). Это принципиально: сервер считает повторное предъявление отозванного *refresh*"=токена кражей и отзывает все сессии, а *React StrictMode* в режиме разработки запускает эффект дважды. После перезагрузки страницы

провайдер аутентификации вызывает тот же *refreshSession*, получает новый *access*”=токен по *cookie* и пользователь остаётся в системе:

```
useEffect(() => {
  setSessionExpiredHandler(() => dispatch({ type: 'anonymous' }));
  authApi
    .restoreSession()
    .then((user) => dispatch({ type: 'authenticated', user }))
    .catch(() => dispatch({ type: 'anonymous' }));
}, []);
```

Ошибки *axios* приводятся к единому виду *{ status, code, message, details }*: ответ сервера, таймаут и отсутствие сети различаются, а ошибки валидации *Joi* (*details*) превращаются в подсказки у полей формы.

3.3 Список задач: загрузка, ошибка, пустой результат

Страница списка загружает задачи хуком *useTasksList*. Он хранит состояние редьюсером (*items*, *total*, *loading*, *error*) и при смене фильтров отменяет предыдущий запрос через *AbortController*, чтобы устаревший ответ не затёр новый:

```
useEffect(() => {
  const controller = new AbortController();
  dispatch({ type: 'load' });
  tasksApi
    .listTasks({ q, status, limit, offset }, controller.signal)
    .then((page) => dispatch({ type: 'loaded', page }))
    .catch((error) => {
      if (!isCancelled(error)) {
        dispatch({ type: 'failed', error: normalizeError(error) });
      }
    });
  return () => controller.abort();
}, [q, status, limit, offset, reloadKey]);
```

Три состояния отображаются явно, в порядке приоритета: ошибка с кнопкой повтора, индикатор первой загрузки, пустой список (с разным текстом для пустой базы и для пустого результата поиска) и сам список:

```
let body;
if (error) {
  body = <ErrorMessage error={error} onRetry={reload} />;
```

```
} else if (loading && items.length === 0) {
  body = <Spinner label="Загружаем задачи…" />;
} else if (items.length === 0) {
  body = filtered
    ? <EmptyState title="Ничего не найдено" hint="…" />
    : <EmptyState title="Задач пока нет" hint="…">
        <Link to="/tasks/new">Создать задачу</Link>
      </EmptyState>;
} else {
  body = (
    <ul>{items.map((t) => <TaskItem key={t.id} … />)}</ul>
  );
}
```

При повторной загрузке (смена поиска или страницы) прежний список остаётся на экране с пониженной непрозрачностью, поэтому интерфейс не «мигает» индикатором. Вид списка представлен на рисунке 3.1, состояние загрузки — на рисунке 3.2, ошибки — на рисунке 3.3, пустого результата — на рисунке 3.4. Загрузка получена искусственной задержкой ответа списка на 2,5 с, ошибка — обрывом запроса.

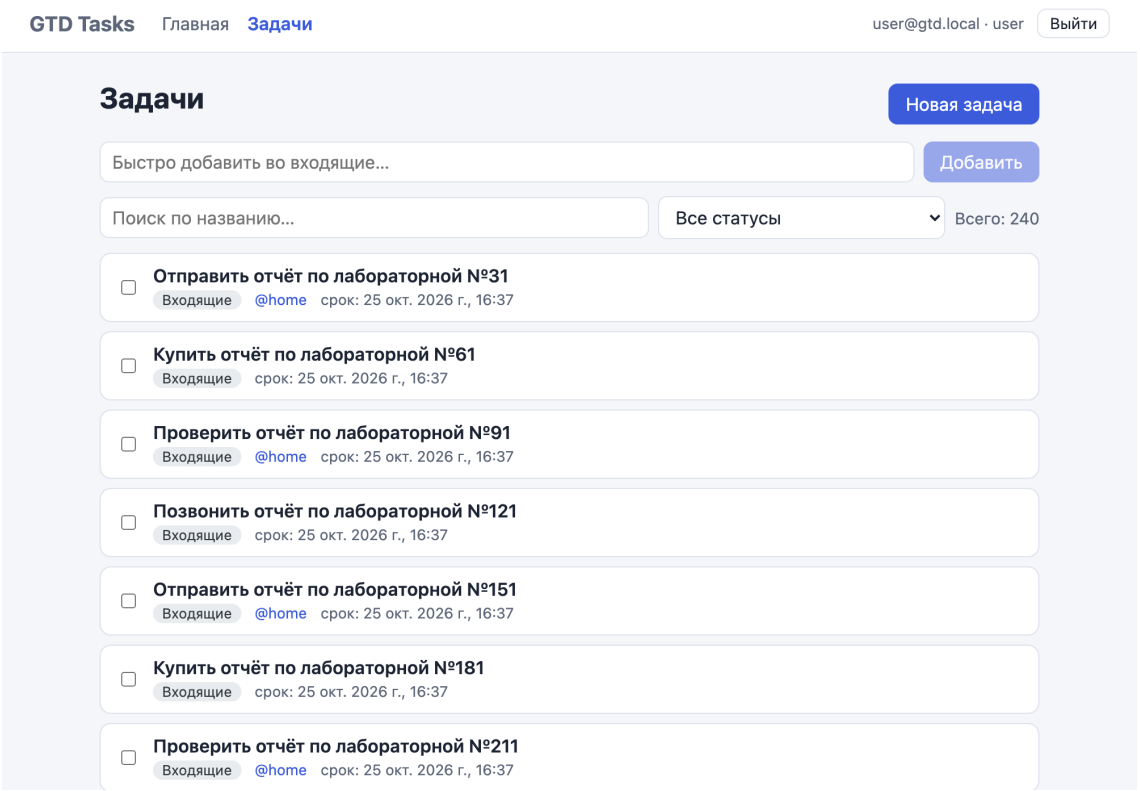


Рисунок 3.1 – Список задач

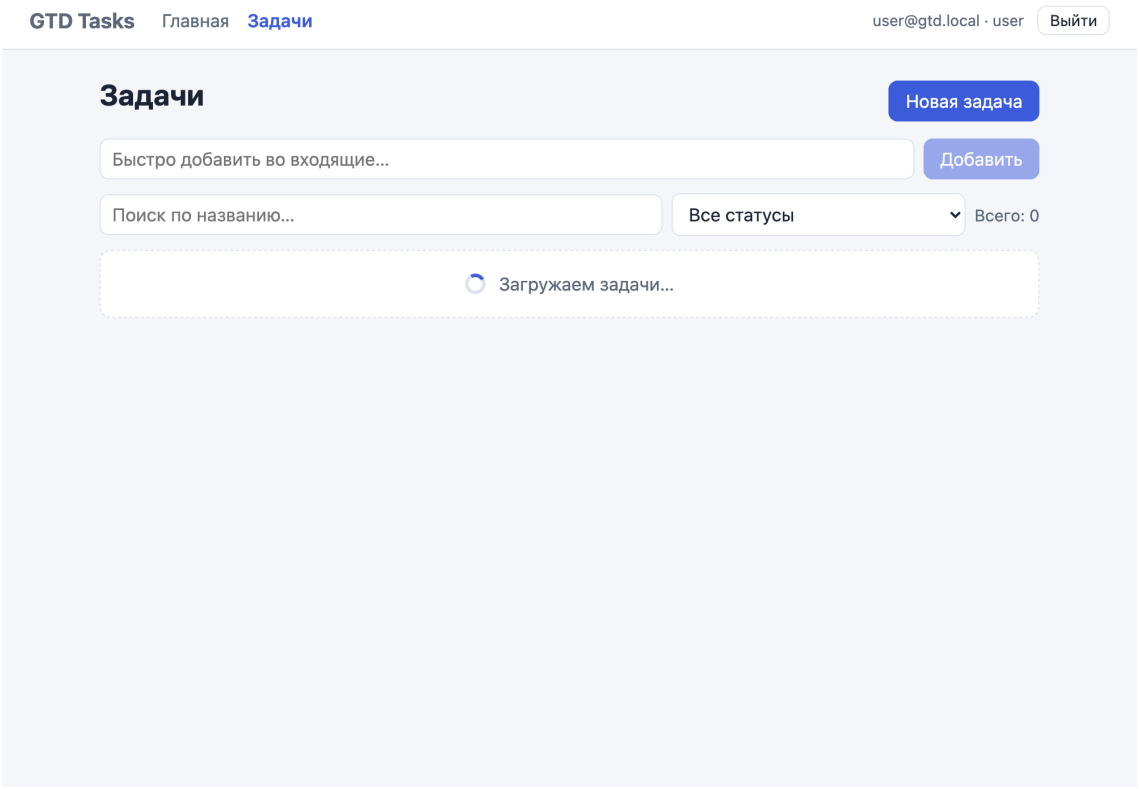


Рисунок 3.2 – Состояние загрузки списка

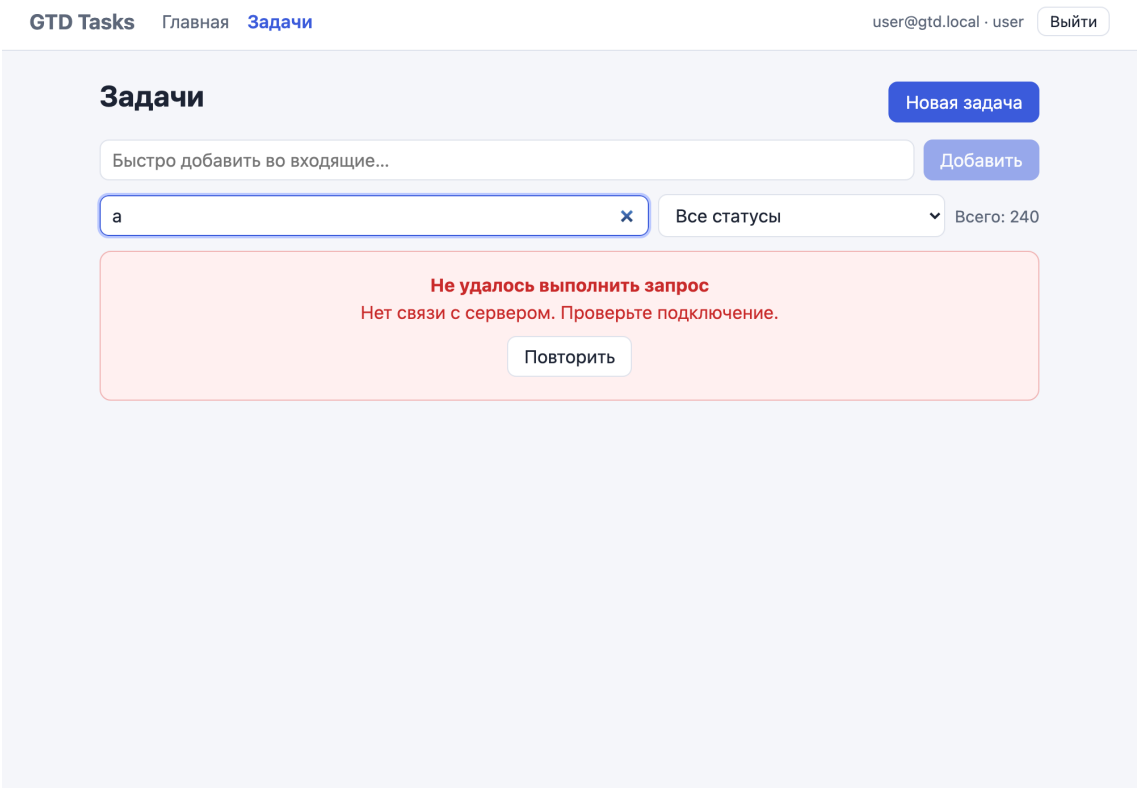


Рисунок 3.3 – Ошибка загрузки списка с кнопкой повтора

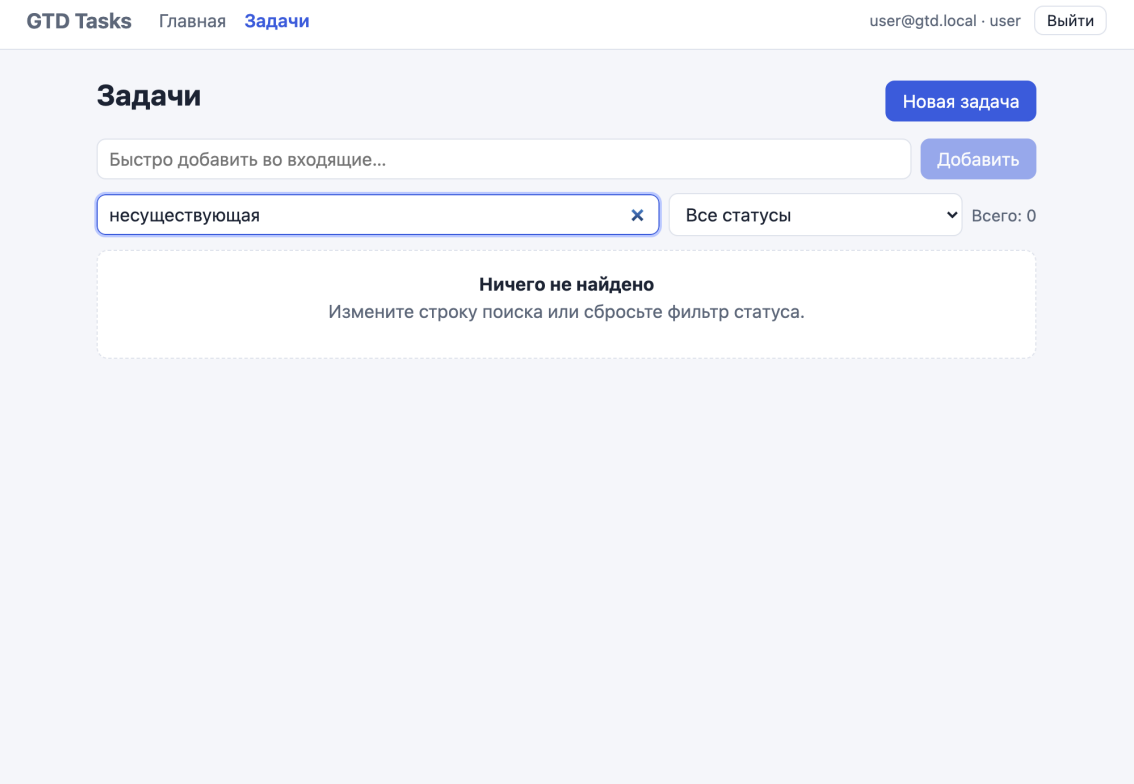


Рисунок 3.4 – Пустой результат поиска

3.4 Страница задачи и форма

Страница задачи параллельно запрашивает задачу и список контекстов (*Promise.all*), чтобы показать имя контекста, а ошибка *404* отображается без кнопки повтора. Кнопка удаления видна только роли *admin*, как требует серверная политика. Страница представлена на рисунке 3.5.

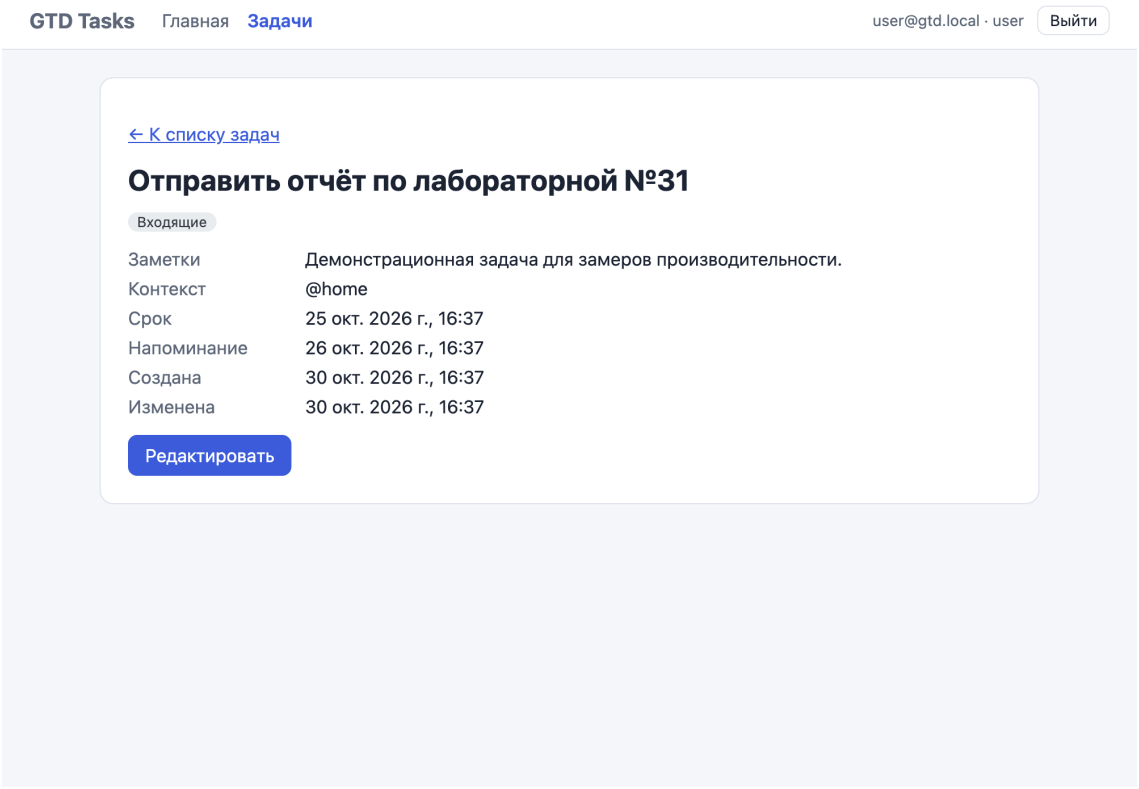


Рисунок 3.5 – Страница задачи

Одна форма служит и для создания, и для редактирования: при наличии параметра *id* она загружает задачу и отправляет *PUT* (полную замену), иначе — *POST*. Перед отправкой выполняется проверка на клиенте, после отправки ошибки сервера *VALIDATION_ERROR* раскладываются по полям:

```
function validate(form) {
  const errors = {};
  if (!form.title.trim()) errors.title = 'Введите название задачи';
  if (form.title.length > 200) {
    errors.title = 'Не более 200 символов';
  }
  if (form.status === 'scheduled' && !form.dueDate) {
    errors.dueDate = 'Для статуса «Запланировано» нужен срок';
  }
  return errors;
}

async function handleSubmit(event) {
  event.preventDefault();
  const found = validate(form);
  setErrors(found);
  if (Object.keys(found).length > 0) return;
  setSaving(true);
```

```
  try {
    const saved = isEdit
      ? await replaceTask(id, toPayload(form))
      : await createTask(toPayload(form));
    navigate('/tasks/' + saved.id);
  } catch (error) {
    const normalized = normalizeError(error);
    setErrors(fieldErrors(normalized));
    setSubmitError(normalized);
    setSaving(false);
  }
}
```

Форма редактирования показана на рисунке 3.6, ошибка проверки (статус «Запланировано» без срока) — на рисунке 3.7, главная страница с напоминаниями — на рисунке 3.8, ответ сервера на неверный пароль — на рисунке 3.9. Скриншоты сняты с работающего приложения (*vite dev*, *API* и *PostgreSQL* на локальной машине) автоматизированным *Chromium*.

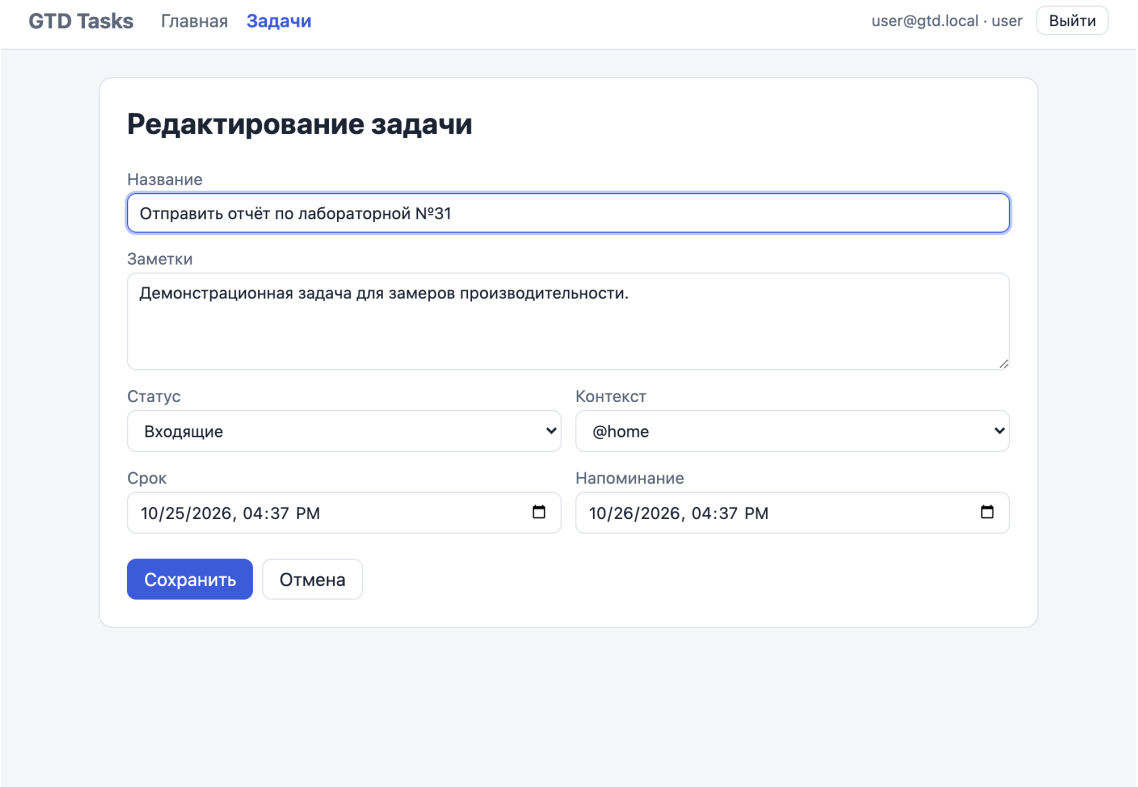


Рисунок 3.6 – Форма редактирования задачи

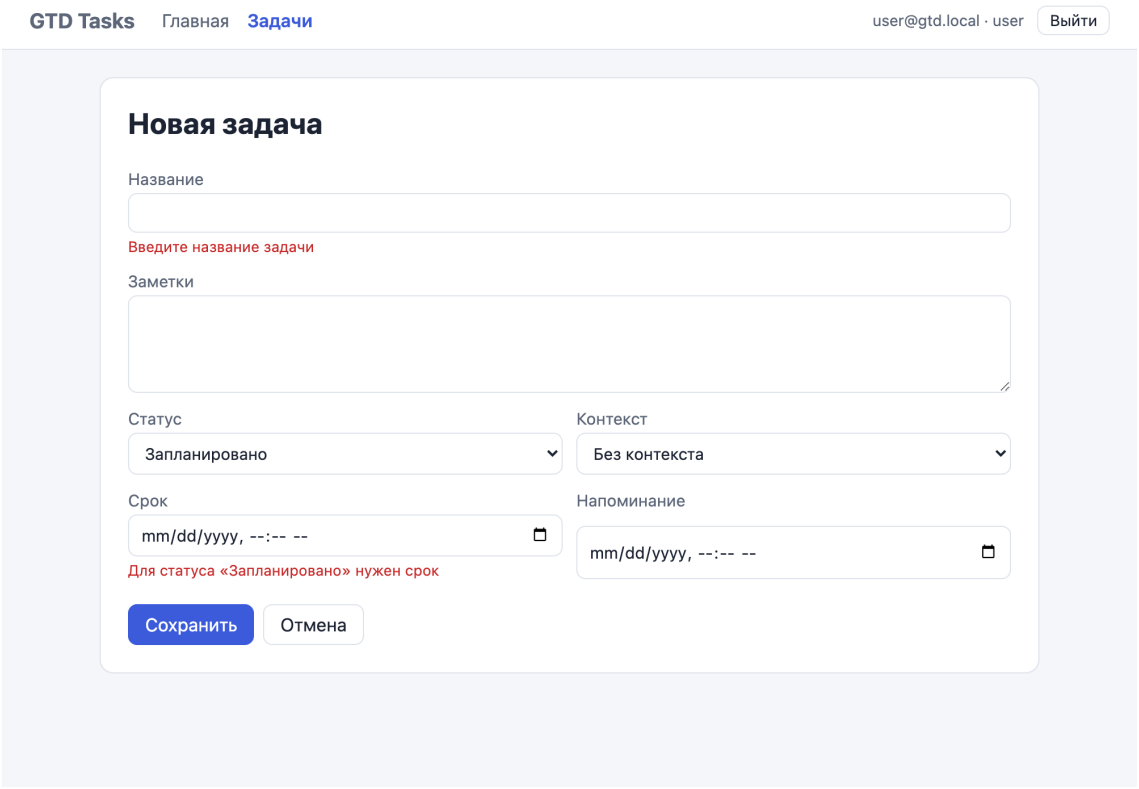


Рисунок 3.7 – Ошибка проверки формы

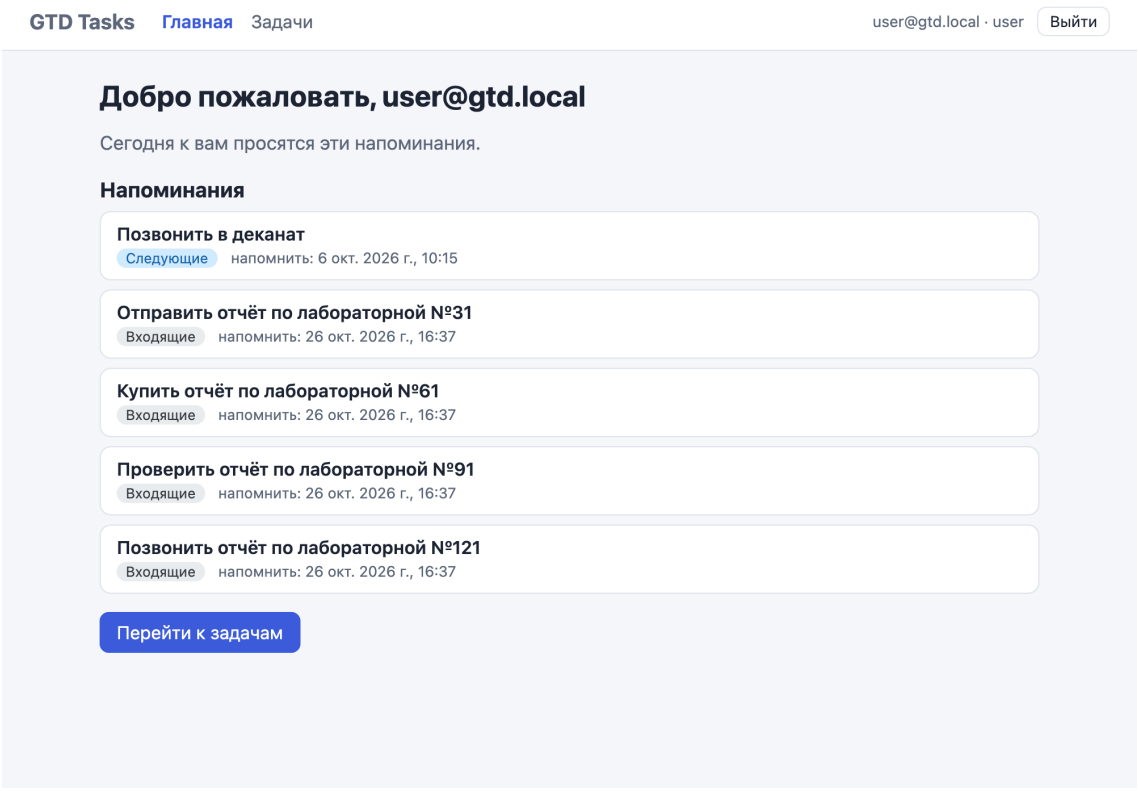


Рисунок 3.8 – Главная страница с напоминаниями

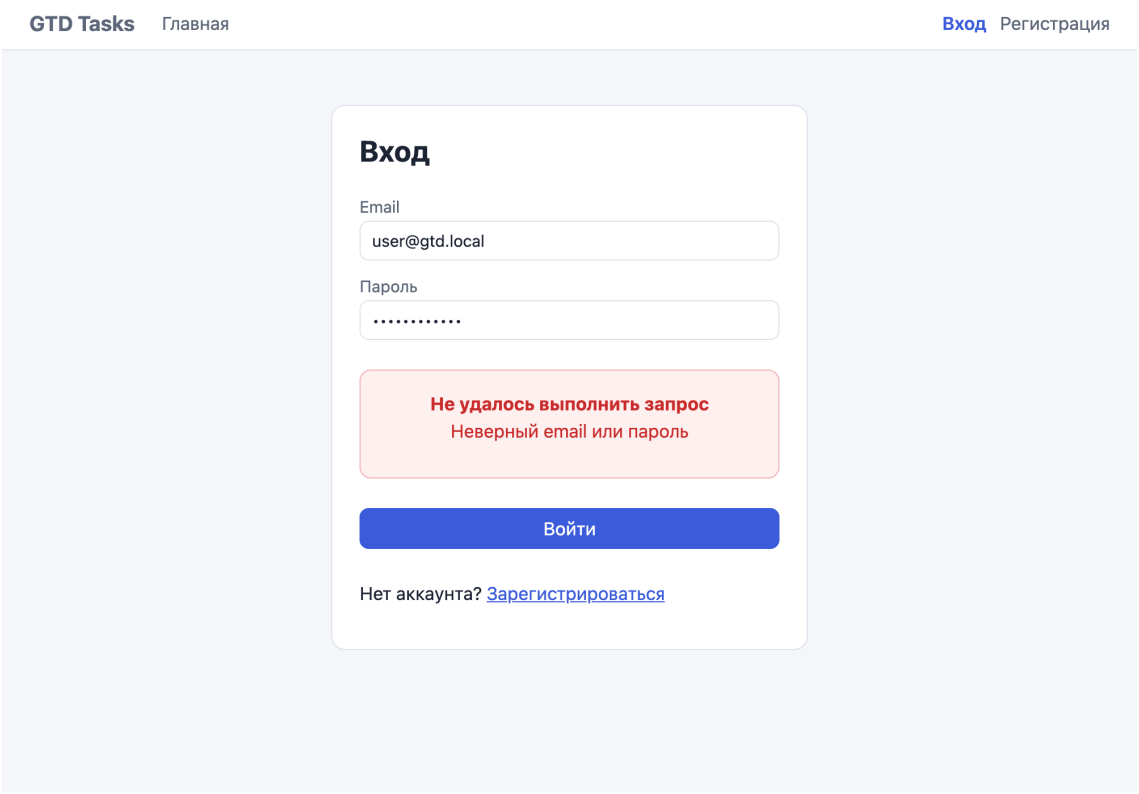


Рисунок 3.9 – Вход с неверным паролем

3.5 Реализованные оптимизации

Реализовано четыре разных приёма плюс опрос напоминаний. Для базовых замеров каждый приём можно отключить параметром адреса (*?debounce=0*, *?memo=0*, *?optimistic=0*); значения читает модуль *perf/flags.js*.

Debounce поиска. Без него каждый введённый символ запускает запрос к серверу. Хук возвращает значение с задержкой 400 мс, а запрос зависит от задержанного значения:

```
export function useDebouncedValue(value, delay) {
  const [debounced, setDebounced] = useState(value);
  useEffect(() => {
    const timer = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(timer);
  }, [value, delay]);
  return debounced;
}

const debouncedSearch = useDebouncedValue(searchInput.trim(), 400);
const q = flags.debounce ? debouncedSearch : searchInput.trim();
```

Поиск выполняет сервер: в *GET /tasks* добавлен параметр *q* (подстрока названия без учёта регистра, спецсимволы *LIKE* экранируются).

Route-based code splitting. Каждая страница подключается через *React.lazy*, *Suspense* с индикатором лежит в общем макете вокруг *Outlet*, поэтому шапка не пропадает во время загрузки чанка:

```
const HomePage = lazy(() => import('./pages/HomePage'));
const TaskListPage = lazy(() => import('./pages/TaskListPage'));
const TaskDetailsPage = lazy(
  () => import('./pages/TaskDetailsPage'));
const TaskFormPage = lazy(() => import('./pages/TaskFormPage'));

<Suspense fallback={<Spinner label="Загружаем страницу…" />}>
  <Outlet />
</Suspense>
```

***React.memo*.** Строка списка обёрнута в *memo*, а колбэки родителя (*create*, *remove*, *toggleDone*) стабилизированы *useCallback*. Без этого каждый введённый символ в поле поиска или быстрого добавления перерисовывает все 20 строк:

```
function TaskItemView({ task, contextName, canDelete,
                        onToggle, onRemove }) {
  countRender('TaskItem');
  ...
}

export const TaskItem =
  flags.memo ? memo(TaskItemView) : TaskItemView;
```

Оптимистичное обновление создания, удаления и отметки «готово» (фрагмент хука *useTasksList*). Новая задача появляется в списке сразу с пометкой «сохраняется», при ошибке сервера изменение откатывается и показывается сообщение:

```
const create = useCallback(async (title) => {
  tempId.current += 1;
  const id = 'tmp-' + tempId.current;
  const draft = { id, title, notes: '', status: 'inbox',
    dueDate: null, contextId: null, pending: true };
  dispatch({ type: 'add', task: draft });
  try {
    const saved = await tasksApi.createTask({ title });
```

```
    dispatch({ type: 'replace', id, task: saved });
  } catch (error) {
    dispatch({ type: 'remove', id });
    fail(error);
  }
}, [fail]);

const remove = useCallback(async (id) => {
  const index = itemsRef.current.findIndex((t) => t.id === id);
  const task = itemsRef.current[index];
  dispatch({ type: 'remove', id });
  try {
    await tasksApi.deleteTask(id);
  } catch (error) {
    dispatch({ type: 'restore', index, task });
    fail(error);
  }
}, [fail]);
```

Опрос напоминаний. На главной странице хук *useDueReminders* раз в 30 с запрашивает *GET /tasks/reminders/due*, пропускает опрос в скрытой вкладке и при уходе со страницы очищает таймер и отменяет запрос.

3.6 Измерения оптимизаций

Все измерения выполнены на локальной машине скриптами из папки *reports/kr22/measure* (*Playwright*, *Chromium*), сырые данные находятся в *reports/kr22/measurements*. Каждый сценарий повторён по пять (бандл — семь) раз для каждого варианта, в таблицах указана медиана.

Debounce. Сценарий: в поле поиска вводится слово «позвонить» (девять символов, 100 мс между нажатиями). Число запросов фиксировалось тремя независимыми способами: событиями *Chromium* (*Network*), счётчиком перехватчика *axios* и журналом сервера; значения совпали во всех десяти прогонах (таблица 3.1, рисунок 3.10). Без *debounce* на сервер уходят запросы с *q=п*, *q=по*, …, *q=позвонить*; с ним — один запрос с полным словом. Каждый запрос без *debounce* дополнительно вызывает предварительный запрос *CORS* (*OPTIONS*), так как адрес у каждого запроса свой.

Таблица 3.1 – Запросы *GET /tasks?q=* при вводе слова из 9 символов

Показатель	Без *debounce*	С *debounce* 400 мс
Запросов от браузера	9	1
Запросов в журнале сервера (*GET*)	9	1
Запросов *OPTIONS* (preflight)	9	1



Рисунок 3.10 – Число запросов поиска до и после *debounce* (по данным *debounce.json*)

React.memo. Измерение выполнено на *profiling*”=сборке (*vite build – mode profiling*, *react-dom/profiling*), так как в обычной сборке *Profiler* не работает. Число рендеров считает счётчик в теле компонента, время — сумма *actualDuration* из *React Profiler API* по всем коммитам (таблица 3.2, рисунок 3.11). При вводе в поле быстрого добавления перерисовывается только родитель (меняется строка ввода), и с *memo* ни одна из 20 строк не перерисовывается. При вводе в поиск остаются 20 рендеров — это отображение полученных результатов. Начальная загрузка даёт 40 и 37 рендеров, потому что после получения списка приходят имена контекстов и строки с контекстом получают новое свойство; на эти рендеры *memo* не влияет. Ненулевое число коммитов с *memo* объясняется тем, что сам родитель и обёртка *Profiler* перерисовываются, а строки — нет.

Таблица 3.2 – Рендеры строки списка и время коммитов (20 строк)

Сценарий	Рендеров без *memo*	Рендеров с *memo*	Время без *memo*, мс	Время с *memo*, мс
Ввод 9 символов в быстрое добавление	180	0	27,3	0,4
Ввод 9 символов в поиск	240	20	33,0	2,4
Первая загрузка списка	40	37	4,1	4,8

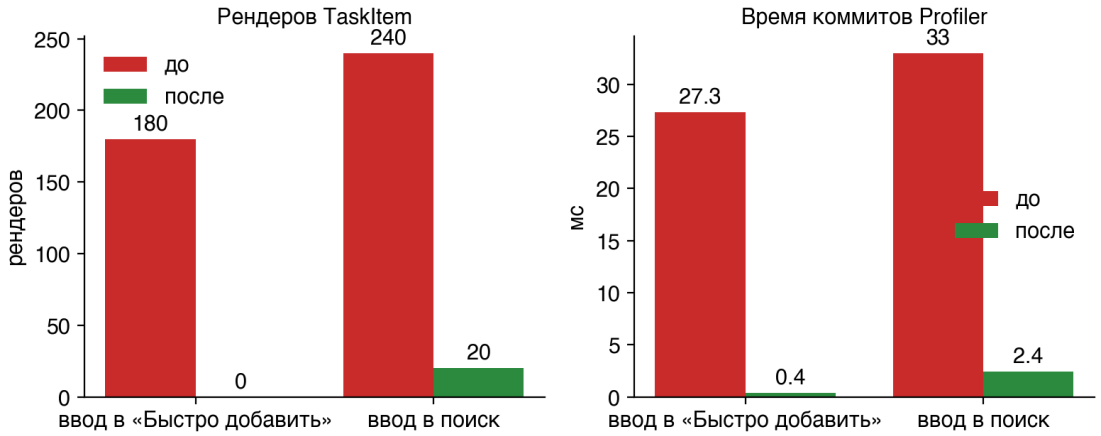


Рисунок 3.11 – Рендеры и время коммитов до и после *React.memo* (по данным *memo.json*)

Code splitting. Размер сборки *vite build* без разделения (все страницы импортируются статически) и с *React.lazy* (таблица 3.3). Вывод *vite* сохранён в *build-baseline.txt* и *build-split.txt*. Стартовый код без разделения — один файл 341,05 КБ (109,88 КБ после *gzip*); с разделением основной файл 276,87 КБ, общий файл *React* 47,08 КБ и семь файлов страниц размером от 0,29 до 7,21 КБ. Список задач (7,21 КБ) и форма (3,73 КБ) не загружаются, пока пользователь на них не перейдёт. Загрузка страницы входа с пустым кэшем: переданных байт *JavaScript* 108,6 КБ против 109,2 КБ, первая отрисовка при эмуляции медленной сети (1,6 Мбит/с, 150 мс, процессор в 4 раза медленнее) 932 мс против 952 мс. Выигрыш на первом экране небольшой (около 0,5% байт и 2% времени): страницы малы по сравнению с библиотеками (*React*, *React Router*, *axios*), которые нужны сразу. Приём оправдан для роста приложения: каждая новая страница не увеличивает стартовый объём.

Таблица 3.3 – Размер *JavaScript* без code splitting и с ним

Показатель	Без разделения	С *React.lazy*
Файлов *JavaScript* в сборке	1	12
Основной файл, КБ (*gzip*)	341,05 (109,88)	276,87 (89,94)
Страница входа: файлов *JS* / КБ по сети	1 / 109,2	4 / 108,6
Первая отрисовка входа, медленная сеть, мс	952	932
Первая отрисовка входа без ограничений, мс	48	48

Оптимистичное обновление. Сценарий: под ролью *admin* создаётся задача, отмечается выполненной и удаляется; задержка сети эмулируется через *CDP* (300 мс на каждый запрос). Измерялось время от клика до изменения в интерфейсе (таблица 3.4, рисунок 3.12). Без оптимизма интерфейс ждёт ответа (более двух кругов обмена из”=за предварительного запроса *CORS*), с оптимизмом реагирует за один кадр.

Таблица 3.4 – Время до изменения интерфейса при задержке сети 300 мс, мс

Операция	Без оптимизма	С оптимизмом
Создание задачи	830	30
Отметка «готово»	815	36
Удаление задачи	831	47

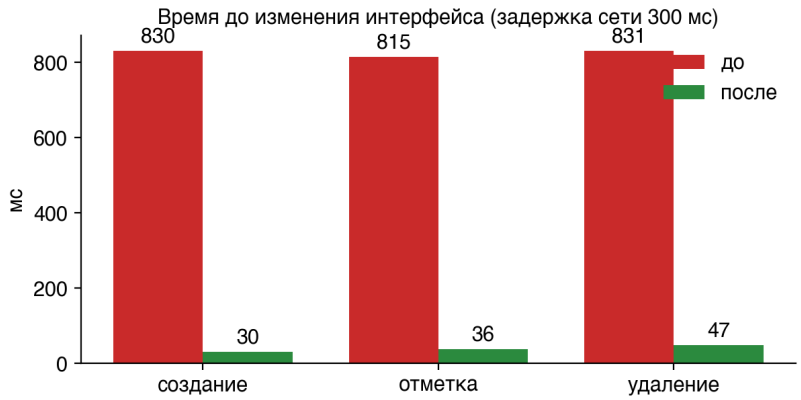


Рисунок 3.12 – Время реакции интерфейса до и после оптимистичного обновления (по данным *optimistic.json*)

3.7 Подготовка бэкенда к развёртыванию

Сервер уже слушал *process.env.PORT*, читал *DATABASE_URL*, *JWT_SECRET*, *JWT_REFRESH_SECRET* и *CORS_ORIGINS*. При подключении клиента найдены и исправлены три проблемы.

1 Список методов *CORS* не содержал *PUT*: браузер отклонял предварительный запрос на замену задачи. Метод добавлен, проверка вынесена в тест *test/cors.test.js* (допустимый источник получает ответ 204 с *PUT*, чужой — 403).

2 *Refresh*”=*cookie* имела *SameSite=Strict*: при размещении клиента и *API* на разных доменах (*Vercel* и *Render*) браузер её не отправит. Добавлена переменная *COOKIE_SAMESITE* (*strict*, *lax* или *none*); значение *none* автоматически включает *Secure*, в *production Secure* включён всегда. Поведение проверяют пять тестов *test/refreshCookie.test.js*.

3 Облачные базы требуют *TLS*: добавлена переменная *DATABASE_SSL*, включающая *ssl* в *Sequelize*.

```
// src/config/env.js
const cookieSameSite =
  readChoice('COOKIE_SAMESITE',
    ['strict', 'lax', 'none'], 'strict');
...
refreshCookie: Object.freeze({
  sameSite: cookieSameSite,
  secure: cookieSameSite === 'none' || nodeEnv === 'production',
}),
```

Все семь тестов проходят командой *npm test*. Для проверки поиска добавлен скрипт *npm run seed:demo*, создающий 240 демонстрационных задач.

3.8 Конфигурации развёртывания

Подготовлены файлы для развёртывания «в один клик» (бэкенд подготовлен в подразделе 3.7); сами развёртывания на *Render* и *Vercel* требуют аккаунтов владельца и в рамках этой работы не выполнялись. Состояние: [будет заполнено после развёртывания].

Файл *render.yaml* (*Blueprint*) описывает веб”=сервис *Node* и базу *PostgreSQL*; строка подключения берётся из базы, секреты генерирует платформа, а значение *CORS_ORIGINS* задаётся вручную после получения адреса фронтенда:

```
databases:
  - name: gtd-postgres
```

```
    plan: free
services:
  - type: web
    name: gtd-task-service
    runtime: node
    buildCommand: npm ci
    startCommand: npm start
    healthCheckPath: /health
    autoDeploy: true
    envVars:
      - { key: NODE_ENV, value: production }
      - { key: TRUST_PROXY, value: "1" }
      - { key: DATABASE_SSL, value: "true" }
      - { key: COOKIE_SAMESITE, value: none }
      - key: DATABASE_URL
        fromDatabase: { name: gtd-postgres,
                        property: connectionString }
      - { key: JWT_SECRET, generateValue: true }
      - { key: JWT_REFRESH_SECRET, generateValue: true }
      - { key: CORS_ORIGINS, sync: false }
```

Файл *client/vercel.json* включает перенаправление всех путей на *index.html* (без него обновление страницы */tasks/5* вернуло бы *404*) и долгое кэширование хешированных файлов:

```
{
  "framework": "vite",
  "buildCommand": "npm run build",
  "outputDirectory": "dist",
  "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }],
  "headers": [{ "source": "/assets/(.*)", "headers": [
    { "key": "Cache-Control",
      "value": "public, max-age=31536000, immutable" }] }]
}
```

Образец переменных клиента находится в *client/.env.example* (*VITE_API_URL*), порядок развёртывания — в разделе «Деплой» файла *README.md*.

4 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

1 Интеграция построена на *axios* с единым экземпляром (*baseURL* из *VITE_API_URL*, *withCredentials*, перехватчики), слоем функций *src/api* и хуками *useEffect*, *useReducer*. Маршрутизация — *React Router*, сборка — *Vite*. Бэкенд — *REST API* на *Express*, обмен в формате *JSON*, ошибки приходят в едином виде с кодом.

2 Состояния описаны явно в *useTasksList* (*loading*, *error*, *items*) и выводятся компонентами *Spinner*, *ErrorMessage* с повтором и *EmptyState*; код приведён в подразделе «Список задач», экраны — на рисунках 3.2, 3.3 и 3.4. Ошибки валидации формы показываются у полей.

3 Реализованы *debounce* поиска, *route-based code splitting*, *React.memo* и оптимистичные обновления, а также опрос напоминаний. Выбор обусловлен узкими местами менеджера задач: поиск при вводе порождает лавину запросов, строки списка перерисовываются при каждом нажатии клавиши, а создание и удаление должны отзываться мгновенно при медленной сети. Виртуальная прокрутка не нужна: страница содержит 20 строк.

4 Тремя способами: счётчиком запросов (события *Chromium*, перехватчик *axios*, журнал сервера), *React Profiler API* на *profiling*”=сборке и счётчиком рендеров, выводом *vite build* и размерами загрузки по *CDP* с эмуляцией сети. Результат: 9 запросов против 1, 180 рендеров против 0, реакция 830 мс против 30 мс; code splitting дал лишь около 0,5% байт на первом экране. Скрипты и сырые данные приложены к репозиторию.

5 Фронтенд предполагается разместить на *Vercel* с корнем *client/*: платформа сама определяет *Vite*, собирает проект командой *npm run build* и перевыпускает его при каждом *push* в подключённую ветку. Конфигурация — *client/vercel.json*. Развёртывание: [будет заполнено после развёртывания].

6 Бэкенд и база описаны в *render.yaml*: веб”=сервис (*npm ci*, *npm start*, проверка */health*) и *PostgreSQL*. Переменные: *NODE_ENV=production*, *TRUST_PROXY=1*, *DATABASE_URL*, *DATABASE_SSL=true*, *JWT_SECRET*, *JWT_REFRESH_SECRET*, *COOKIE_SAMESITE=none*, *CORS_ORIGINS*; *PORT* выдаёт платформа. Развёртывание: [будет заполнено после развёртывания].

7 *HTTPS* обеспечивают платформы размещения; *cookie* с *refresh*”=токеном имеет флаги *HttpOnly*, *Secure* и *SameSite=None* (разные домены). *CORS* разрешает только источники из *CORS_ORIGINS* вместе с *credentials*, чужие получают 403. *Access*”=токен живёт 15 минут и хранится только в памяти, *refresh*”=токен вращается при каждом обновлении, а его повтор отзывает все сессии. Весь вывод в *React* экранируется автоматически, *dangerouslySetInnerHTML* не применяется.

8 *Code splitting* — разделение кода приложения на части, которые загружаются по мере необходимости, а не одним файлом. *React.lazy(() => import('./pages/X'))* превращает динамический импорт в компонент, а *Suspense* показывает запасной интерфейс, пока чанк грузится. Сборщик создаёт для каждой страницы отдельный файл (см. таблицу 3.3).

9 Мемоизация сама стоит памяти и времени на сравнение свойств, поэтому в большом приложении она может не окупаться. Типичные проблемы: новые объекты, массивы и функции в свойствах каждый раз ломают *memo*; неполный список зависимостей *useMemo* и *useCallback* даёт устаревшие значения; мемоизация усложняет код и скрывает настоящие причины лишних рендеров. Применять её нужно после измерения: здесь она оправдана замером (180 рендеров против 0).

10 *Debounce* откладывает действие до паузы в событиях и выполняет его один раз после последнего (поиск при вводе), а *throttle* выполняет действие не чаще одного раза за интервал (прокрутка, изменение размера окна). Для поиска использован *debounce*, потому что нужен только итоговый запрос; *throttle* в приложении не применён.

11 Виртуальная прокрутка (*react-window*) создаёт в *DOM* только строки, попадающие в окно просмотра, и подменяет их при прокрутке; остальные существуют как данные. Она нужна при тысячах элементов, когда *DOM* становится тяжёлым. В работе не применена: список постраничный, по 20 строк.

12 Библиотеки кэширования (*React Query*, *SWR*) не использовались, ручного кэша ответов нет. Косвенно работают *Cache-Control: immutable* для файлов сборки на *Vercel*, кэширование ответа *preflight* (600 с) и общий запрос обновления токена. При развитии стоило бы подключить *React Query*.

13 В *Vercel* открывается *Settings → Environment Variables*, добавляется *VITE_API_URL* со значением адреса бэкенда без слеша на конце (например, *https://gtd-task-service.onrender.com*) для окружений *Production* и *Preview*, после чего проект пересобирается: переменные *Vite* подставляются при сборке, а не во время работы. Доступны только переменные с префиксом *VITE_*, секретов в них хранить нельзя. Значения для этой работы: [будет заполнено после развёртывания].

14 Использованы *React Profiler API* (компонент *Profiler* на *profiling*"=сборке), счётчик рендеров, *Chromium DevTools Protocol* для эмуляции сети и процессора, события *Network* и журнал сервера. Аудит *Lighthouse* и расширение *React DevTools* в работе не запускались; их заменяют перечисленные программные измерения.

15 Оптимистичное обновление применено к созданию, удалению и отметке задачи: интерфейс меняется сразу, запрос идёт в фоне, при ошибке вы-

полняется откат. Фрагмент кода приведён в подразделе «Реализованные оптимизации» (функции *create* и *remove*), эффект — в таблице 3.4.

16 При большем времени: выполнить развёртывание и аудит *Lighthouse* на реальных адресах, подключить *React Query* для кэша и повторных попыток, добавить виртуальную прокрутку и бесконечную прокрутку, *WebSocket* вместо опроса напоминаний, тесты клиента (*Vitest*, *Testing Library*) и сквозные тесты, вынос *refresh*”=токена в схему с защитой от *CSRF* (отдельный заголовок), а также режим офлайн.

ЗАКЛЮЧЕНИЕ

В ходе контрольной работы разработан клиент на *React* и *Vite*, интегрированный с *REST API* сервиса планирования задач по методу *GTD*: вход и регистрация с *JWT* (токен доступа в памяти, обновление по *httpOnly cookie*), список, страница и форма задачи, явная обработка загрузки, ошибок и пустого списка.

Реализованы и измерены четыре оптимизации. *Debounce* сократил число запросов поиска с 9 до 1, *React.memo* — число рендеров строк с 180 до 0 при вводе в поле, оптимистичные обновления — время реакции с 830 до 30 мс при задержке сети 300 мс. *Code splitting* уменьшил основной файл с 341 до 277 КБ, но на первом экране выигрыш составил около 0,5% байт. Бэкенд подготовлен к развёртыванию: исправлены *CORS* для *PUT* и параметры *cookie* для разных доменов, добавлены тесты, *render.yaml* и *client/vercel.json*. Развёртывание на облачных платформах и ссылки на работающее приложение: [будет заполнено после развёртывания].